

Bluechip: Evidence-First Agents for Review Simulation and Recommendation

DSN x BCT LLM Agent Challenge - Task A and Task B Solution Paper

Abstract

Bluechip is an evidence-first user intelligence agent for the DSN x BCT LLM Agent Challenge. The brief asks for two capabilities: Task A, an agent that predicts a user's star rating and simulates a review for an unseen product; and Task B, an agent that recommends personalized items while handling cold-start, cross-domain, and conversational scenarios. We implement both tasks as serving heads over the same user-profile, item-profile, retrieval, ranking, generation, validation, and tracing system.

The central design decision is that the LLM is not the ranking oracle. Deterministic and evaluated components handle rating prediction, candidate retrieval, ranking, and validation. LLM-backed generation is optional and late in the pipeline: it writes the final review or explanation after the system has already made the measurable decision. This keeps the submission reproducible without provider keys, while still allowing richer language when OpenRouter, DeepSeek, or OpenAI credentials are supplied.

Current bounded evidence shows Task A serving is rating-first with a documented 5,000-example all-category RMSE gate of 1.2654. A final 25-example all-category generation smoke had 1.0 validation consistency, rating mention, item mention, and sentiment alignment under deterministic fallback. Task B's measured all-category candidate recall remains the main bottleneck: the stronger logged 100-example/1000-candidate gate has candidate Recall@1000 0.34, HitRate@10 0.10, and NDCG@10 0.0766; the final 50-example/100-candidate smoke had HitRate@10 0.06 and NDCG@10 0.0471. Cross-domain candidate Recall@1000 is the strongest measured Task B slice at 0.5484. A later candidate-recall-only fast proof showed that objective alignment matters: under a positive-recommendation target, Recall@1000 rose to 0.3986, sparse Recall@1000 to 0.3973, and cross-domain Recall@1000 to 0.6081, but final HitRate@10 and NDCG@10 still require a same-target ranker run. Neural FAISS retrieval is now wired with durable item-id mapping and passes a bounded smoke, but it is not presented as a ranking-quality win until larger same-slice evals prove lift.

1. Problem Framing

The competition is not only a text-generation problem. Review simulation and recommendation both require a model of how a person behaves: what they value, what they dislike, how strict their ratings are, how context changes their choices, and what evidence should be trusted. A prompt-only solution can sound plausible but is difficult to evaluate. It cannot reliably compute RMSE, candidate recall, NDCG@10, or source-level ablations.

Bluechip therefore treats the challenge as a user intelligence system with two outputs:

Task A: user and item evidence -> rating prediction -> review plan -> review text -> validation

Task B: user and context evidence -> candidate retrieval -> ranking -> explanation -> trace

This separation lets us evaluate the measurable parts before asking a language model to produce prose. It also makes failures visible: if Task B misses the held-out product, we can tell whether candidate generation failed, ranking failed, or generation merely explained a bad recommendation.

The implementation is informed by established recommender-system patterns. The two-stage retrieval/ranking shape follows the same practical separation used by large-scale systems such as YouTube-style candidate generation and ranking. Item-to-item and co-engagement retrieval are included because Amazon-style item similarity remains a strong sparse-data baseline. The review-aware profile layer is inspired by DeepCoNN/NARRE-style use of review text, but implemented as transparent aspect and evidence extraction rather than a neural review encoder. The generation layer follows RAG and explainable recommendation practice: retrieve bounded user/item evidence, decide with deterministic scoring, then ask an LLM or fallback template to express the result. We considered BPR/ALS, Wide & Deep, LightGCN, SASRec, HSTU, PETER, and PEPLER; under the deadline, we implemented the pieces that directly improved measurable retrieval, rating, explanation, and reproducibility, and left heavier neural models as future work.

2. System Architecture

The local implementation is a FastAPI application with typed Pydantic request and response schemas. The same internal services support both tasks:

Raw reviews and metadata
-> ingestion and temporal splits
-> user profile and item profile extraction
-> aspect and Nigerian-context evidence extraction
-> retrieval indexes and evidence graph
-> Task A rating prediction / Task B ranking
-> grounded generation
-> validation and trace logging
-> API and browser demo

Important modules:

API - Local implementation: app/api/routes.py, app/main.py
Task orchestration - Local implementation:
app/serving/orchestrators/review_simulation.py,
app/serving/orchestrators/recommendation.py
Profiling - Local implementation: app/services/profiling/
app/services/intelligence/
Retrieval - Local implementation: app/services/retrieval/
Ranking and rating - Local implementation: app/services/ranking/
Generation - Local implementation: app/services/generation/
Validation - Local implementation: app/services/validation/
Nigerian context - Local implementation: app/services/nigerian/
Conversation state - Local implementation: app/services/conversation/
Observability - Local implementation: app/stores/trace_store.py,
/api/metrics, /api/traces
Evaluation - Local implementation: eval/

The production mapping is straightforward: local JSONL files become an object-store data lake, local feature and model registries become managed registries, local retrieval artifacts become ANN/vector/graph services, and the in-process ranker becomes a dedicated ranking service. The important design boundary is already present locally: offline data and evaluation produce versioned artifacts; online serving loads those artifacts, handles fallbacks, and records traces.

3. Task A: Rating-First Review Simulation

Task A input is a persona or history plus a target product. The output is a predicted rating, generated review, confidence, user signals, item signals, validation result, and trace.

The serving flow is:

1. Build a UserProfile from visible history and persona text.
2. Build an ItemProfile from target item metadata and summary.
3. Predict the rating using the promoted Task A serving policy or deterministic fallback.
4. Build a review plan from the predicted rating, user signals, item signals, and target item facts.
5. Generate review text through the configured provider or deterministic template fallback.
6. Validate rating-review consistency and grounding.
7. Return the response and trace.

The rating-first design is deliberate. If a generator writes the review first, it can drift into text that implies a different score from the measured rating. Bluechip fixes the star decision before generation, so the review has to explain the rating rather than invent it. This mirrors the challenge scoring, where rating accuracy and review quality are separate criteria.

Current Task A evidence:

Serving strategy - Value or status: Rating-first generation
Latest documented all-category RMSE gate - Value or status: 1.2654 on 5,000 examples
Final all-category generation smoke - Value or status: 25 examples; validation consistency 1.0, rating mention 1.0, item grounding 1.0, sentiment alignment 1.0
Generation checks - Value or status: Provider failure rate, fallback rate, validation consistency, rating mention, item grounding, sentiment alignment, ROUGE-L F1, unigram F1
Fallback behavior - Value or status: Runs without external API keys

The implementation also supports provider-backed review generation with OpenRouter, DeepSeek, or OpenAI. External provider evaluation defaults to restricted data handling, with redact or allow required for non-sample data export.

4. Task B: Retrieval Before Ranking

Task B input is a persona or history, optional current context, candidate items, locale, and limit. The output is ranked recommendations with reasons, tradeoffs, source attribution, score components, diagnostics, and trace.

The serving flow is:

1. Build or load a user profile.
2. Detect cold-start or sparse-history conditions.
3. Retrieve candidates from multiple heads.
4. Preserve candidate source attribution and retrieval scores.
5. Rank candidates with a hybrid multi-objective scorer.
6. Generate grounded explanations after ranking.
7. Return candidate diagnostics and a trace.

Candidate sources include co-visitation, user-neighbor collaborative retrieval, review-term retrieval, lexical item-neighbor retrieval, evidence graph retrieval, BM25 profile/context retrieval, category-affinity popularity, global popularity fallback, and vector diagnostics. This is a local multi-head retrieval shape: each head contributes a different kind of behavioral or semantic evidence, and the evaluator reports source-level performance.

Ranking blends explicit components: preference match, context match, category match, aspect match, sequential signal, evidence graph signal, Nigerian-context match, collaborative signal, retrieval match, source diversity, popularity, novelty, item quality, confidence, and penalties for disliked or context-mismatched evidence.

Current bounded Task B metrics:

hybrid_candidate_recall@50 - Value: 0.13 | Interpretation: Early candidate pool remains weak.
hybrid_candidate_recall@100 - Value: 0.18 | Interpretation: Recall is the main bottleneck.
hybrid_candidate_recall@1000 - Value: 0.34 | Interpretation: Best current overall candidate recall signal.
hybrid_ranker_hit_rate@10 - Value: 0.10 | Interpretation: Current top-10 ranking gate.
hybrid_ranker_ndcg@10 - Value: 0.0766 | Interpretation: Current top-10 ranking gate.
Sparse candidate recall@1000 - Value: 0.3611 | Interpretation: Sparse-user handling is credible but not solved.
Cross-domain candidate recall@1000 - Value: 0.5484 | Interpretation: Strongest measured Task B slice.
Vector source recall - Value: 0.0 | Interpretation: Diagnostic only; not claimed as a quality improvement.

A 24 May 2026 candidate-recall fast proof rebuilt retrieval artifacts with rating-weighted all-interaction evidence in an isolated directory. On the full all-interactions target, the result was mixed: Recall@1000 improved to 0.362 and cross-domain Recall@1000 to 0.5752, but Recall@100 fell to 0.1589. On the positive-recommendation target, which excludes held-out ratings below 4, retrieval passed the diagnostic gates: Recall@50 0.151, Recall@100 0.1823, Recall@1000 0.3986, sparse Recall@1000 0.3973, and cross-domain Recall@1000 0.6081. This is not counted as a final ranking promotion because it did not rerun the top-10 ranker metrics.

Final validation also ran a fresh 50-example all-category smoke at candidate-limit 100: hybrid HitRate@10 0.06, NDCG@10 0.0471, candidate Recall@50 0.08, candidate Recall@100 0.12, and cross-domain candidate Recall@100 0.2857. A separate 5-example all-category neural FAISS smoke confirmed the prebuilt 188,236-vector index loads with the companion item-id map and contributes neural_vector candidates; that tiny slice had zero held-out hits and is used only as runtime validation, not as a quality claim.

These numbers are intentionally conservative. The submission does not claim that semantic vectors solved recommendation. Instead, it shows the exact bottleneck: candidate generation must improve before more sophisticated ranking models can matter.

5. Nigerian Contextualization

The brief awards additional marks for agents that behave and sound Nigerian. We implement this as evidence-aware contextualization, not decorative phrasing.

The Nigerian context layer detects and uses signals such as:

- city and regional cues including Lagos, Abuja, Kano, Ibadan, Port Harcourt, Aba, Onitsha, Enugu, Jos, and Calabar;
- price sensitivity in a Naira economy;
- delivery and logistics concern;
- social proof and community recommendation behavior;
- quality skepticism around "original" versus fake products;
- market, mall, and online-shopping distinctions;
- optional Nigerian English and light pidgin voice when the persona or locale supports it.

For Task A, Nigerian voice is applied only after rating prediction and only when locale or persona evidence justifies it. For Task B, Nigerian context contributes to score components when the user context makes it relevant, for example affordable products, delivery reliability, gift suitability, or quality-for-money concerns.

This keeps the system from forcing Nigerian markers into every output. A Lagos student looking for an affordable dinner spot should sound different from a Kano trader buying business inventory or an Abuja civil servant seeking a reliable gift.

6. Experiments and Ablations

The evaluation suite is designed around fixed, replayable scripts:

```
eval/eval_task_a.py - Purpose: Rating accuracy for Task A
eval/train_task_a_model.py - Purpose: Train and select Task A rating artifacts
eval/promote_task_a.py - Purpose: Promote Task A serving policy after a fixed gate
eval/eval_task_a_generation.py - Purpose: Review generation quality and provider/fallback diagnostics
eval/eval_task_b.py - Purpose: Candidate recall, HitRate@K, NDCG@K, sparse and cross-domain slices
eval/aggregate_task_b_reports.py - Purpose: Aggregate deterministic Task B eval shards
eval/report_task_b_from_row_cache.py - Purpose: Derive alternate Task B target-mode reports from cached eval rows
eval/create_task_b_contextual_eval.py - Purpose: Human-eval packet for contextual relevance
eval/eval_evidence_intelligence.py - Purpose: Aspect and evidence-layer checks
```

Ablations already supported or documented include:

- Task A with and without promoted rating artifacts.
- Task A generation with deterministic fallback versus provider-backed text.
- Task B popularity and category baselines versus hybrid retrieval/ranking.
- Candidate Recall@50/100/1000 separated from top-rank HitRate@10 and NDCG@10.
- Sparse-user and cross-domain slices separated from aggregate metrics.
- All-interactions versus positive-recommendation target modes separated for Task B retrieval diagnostics.
- Source-level diagnostics for retrieval heads, including neural FAISS and vector retrieval as diagnostic paths.
- Contextual human-eval examples with source traces.

The most important ablation result is negative: vector and neural retrieval are implemented and observable, but current tracked slices do not prove ranking lift. That result is useful because it prevents the submission from overclaiming neural semantics and points future work toward learned retrieval only after fixed same-slice evals prove lift.

7. Reproducibility

The repository is designed to run without secrets. A judge can install dependencies, run the API, inspect OpenAPI docs, run sample evals, and build the Docker container.

Core commands:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -e ".[dev]"
uvicorn app.main:app --reload
```

Docker:

```
docker compose up --build
```

Useful local URLs:

```
http://127.0.0.1:8000/ui/
http://127.0.0.1:8000/docs
http://127.0.0.1:8000/api/health
http://127.0.0.1:8000/api/metrics
http://127.0.0.1:8000/api/traces
```

Core validation:

```
ruff check .
pytest
python -m compileall app eval scripts tests
```

Sample eval:

```
make eval
```

Full-data evals require local Amazon Reviews 2023 artifacts under `data/processed/all_categories`, which are intentionally not committed. The README documents the download, ingestion, indexing, Task A promotion, Task B evaluation, and contextual human-eval commands.

8. Limitations and Next Work

The current system is submission-ready, but it is not presented as solved personalization.

Known limitations:

- Candidate generation is still the main Task B bottleneck, especially at Recall@50 and Recall@100.
- HitRate@10 and NDCG@10 are modest and should be treated as baseline gates, not final product performance.
- Task B contextual human evaluation is scored on 20 examples; Task A has a judge-facing review pack, while the official behavioural score remains judge-run.
- Neural/vector retrieval is wired and reproducible, but it is not a promoted quality signal until larger same-slice evals show lift.
- Local feature/model registries are production-shaped abstractions, not managed cloud services.
- LLM output improves readability but should remain downstream of deterministic scoring and validation.

Highest-value next steps:

1. Run the same-target positive-recommendation ranker pass and compare HitRate@10 and NDCG@10 against the current 0.10 / 0.0766 gate.
2. Improve candidate Recall@50/100 while preserving cross-domain Recall@1000.
3. Train a true learned multi-head retriever only after fixed retrieval ablations show a target signal.
4. Collect human labels for behavioral fidelity and contextual relevance.
5. Move local registry, trace, feature, and retrieval artifacts into managed production services.

9. Rubric Alignment

Task A review and rating simulation - Bluechip response: `/api/simulate-review` predicts rating, generates review, validates consistency, returns trace.

Task B personalized recommendation - Bluechip response: `/api/recommend` retrieves, ranks, explains, and returns candidate diagnostics.

Cold-start and cross-domain - Bluechip response: Persona-only and sparse-user handling are implemented and measured; cross-domain candidate Recall@1000 is 0.5484.

Multi-turn scenarios - Bluechip response: `/api/conversation/turn` and feedback endpoints maintain conversation state.

Solution paper - Bluechip response: This paper explains architecture decisions, experiments, ablations, limitations, and next work.

Reproducible code - Bluechip response: FastAPI app, Dockerfile, docker-compose, sample data, eval scripts, README, and deterministic fallback are included.

Nigerian contextualization - Bluechip response: Dedicated Nigerian context and voice services use locale/persona evidence for scoring and generation.

Clean repository - Bluechip response: Modular services, typed schemas, tests, linting, metrics, traces, docs, and sample artifacts are included.

Bluechip's core claim is disciplined rather than exaggerated: user modeling and recommendation should be measurable before they are eloquent. The system therefore prioritizes traceable evidence, evaluation gates, and honest failure analysis, then uses LLMs to express the result clearly.